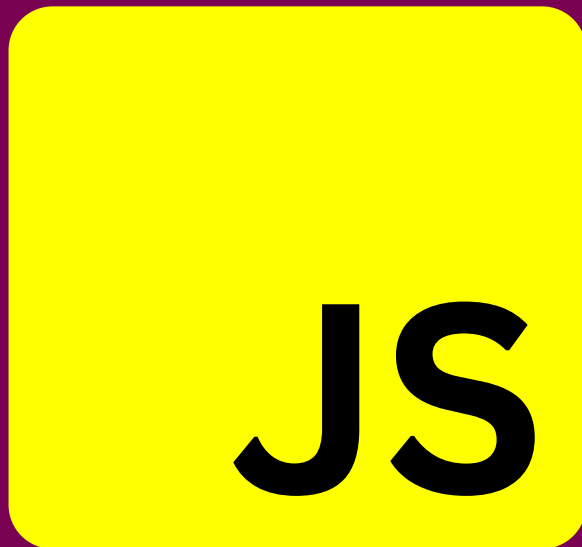


info@ecodrix.com

”



Let _{Vs} Var _{Vs} Const

Explained with Clear Visuals!

FOLLOW FOR MORE



Swipe

Scope: Function vs Block

- `var` is function-scoped, while `let` and `const` are block-scoped.

```
1  function scope() {
2    if (true) {
3      var x = 10;
4      let y = 10;
5      const z = 10;
6    }
7
8    if (false) {
9      // this block doesn't run
10     var xx = 10;
11     let yy = 10;
12     const zz = 10;
13   }
14
15   console.log(x); // ✓ 10 – function scoped (var)
16   console.log(y); // ✗ ReferenceError – block scoped (let)
17   console.log(z); // ✗ ReferenceError – block scoped (const)
18
19   console.log(xx); // ✓ undefined – declared, but block didn't run
20   console.log(yy); // ✗ ReferenceError
21   console.log(zz); // ✗ ReferenceError
22 }
23 scope();
```

Swipe

FOLLOW FOR MORE



Hoisting + TDZ (Temporal Dead Zone)

- `var` is hoisted and initialized with `undefined`
- `let` and `const` are hoisted but not initialized – so accessing them before declaration throws a TDZ error.



```
1
2 console.log(a); // ✓ undefined (hoisted var)
3 var a = 5;
4
5 console.log(b); // ✗ ReferenceError (TDZ)
6 let b = 10;
7
8 console.log(c); // ✗ ReferenceError (TDZ)
9 const c = 15;
10
```

FOLLOW FOR MORE



Swipe

Redeclaration & Reassignment

- **var**: can redeclare + reassign
- **let**: can reassign but not redeclare
- **const**: cannot redeclare or reassign



```
1  var a = 1;
2  var a = 2; // Allowed
3  a = 3; // Reassignment allowed
4
5  let b = 10;
6  // let b = 20; // SyntaxError (redeclaration not allowed)
7  b = 30; // reassignment allowed
8
9  const c = 100;
10 // const c = 200; // SyntaxError
11 c = 300; // TypeError (cannot reassign)
12
```

Swipe

FOLLOW FOR MORE



Closures in Loops (var vs let)

- **var** has function scope — all timers share the same
- **let** is block-scoped — each iteration gets a new



```
1  for (var i = 0; i < 5; i++) {  
2      setTimeout(function () {  
3          console.log(i); // 5 5 5 5 5  
4      }, 1000);  
5  }  
6  
7  for (let j = 0; j < 5; j++) {  
8      setTimeout(function () {  
9          console.log(j); // 0 1 2 3 4  
10     }, 1000);  
11 }  
12
```

FOLLOW FOR MORE



Swipe

const with Objects and Arrays

- **const** means no reassignment — but internal properties can change



```
1  const person = {
2    name: "John",
3    age: 30,
4    hobbies: ["reading", "coding"],
5  };
6  person.name = "Jane";
7  //person = { name: "Jane", age: 30, hobbies: ["reading", "coding"] };
8  person.age = 35;
9  //person = { name: "Jane", age: 35, hobbies: ["reading", "coding"] };
10 person.hobbies.push("traveling");
11 /*person = {
12   name: "Jane",
13   age: 35,
14   hobbies: ["reading", "coding", "traveling"],
15 };*/
16
17 //person = {}; // TypeError
18
19 const numbers = [1, 2, 3, 4, 5];
20 numbers.push(6); // [1, 2, 3, 4, 5, 6]
21 numbers[0] = 7; // [7, 2, 3, 4, 5, 6]
22 numbers[2] = 8; // [7, 2, 8, 4, 5, 6]
23 //numbers = []; // TypeError
```

Swipe

FOLLOW FOR MORE



Variable Shadowing

- Inner `let/const` can shadow outer variables of the same name

```
1  let message = "outside";
2
3  function showMessage() {
4      let message = "inside"; // shadows outer
5      console.log(message); // "inside"
6  }
7
8  showMessage();
9  console.log(message); // "outside"
10
```

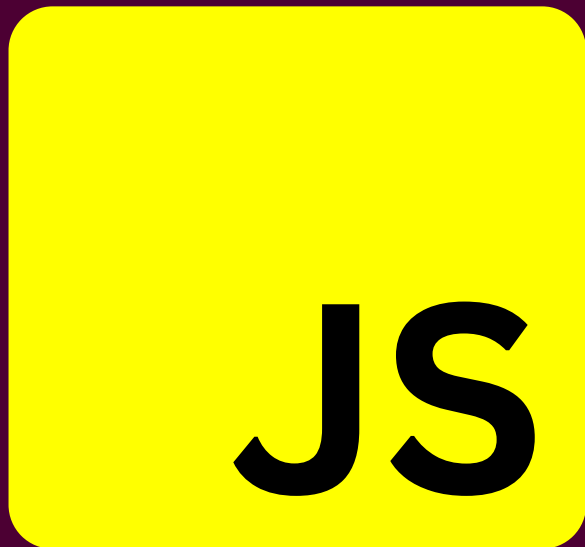
FOLLOW FOR MORE



Swipe

info@ecodrix.com

”



Thank You For Reading

**Follow us for more daily JavaScript &
SaaS dev tips!**

